Recursive algorithms

One strategy for solving problems is:

- If the problem is too big to solve directly, reduce it to smaller problems of the same type, and apply the same procedure to those.

That is, the algorithm calls itself: it is *recursive*. Such algorithms will be the topic of this session, and in week 7 we'll see how this strategy has been used to obtain fast sorting algorithms. But first we'll review methods.

Understanding method calls

We'll begin with another look at methods, which were covered in session 3 of the Java module last term.

Tracing method calls

Consider the following simple algorithms:

Algorithm Square(x)

```
RETURN x * x
```

Algorithm Cube(x)

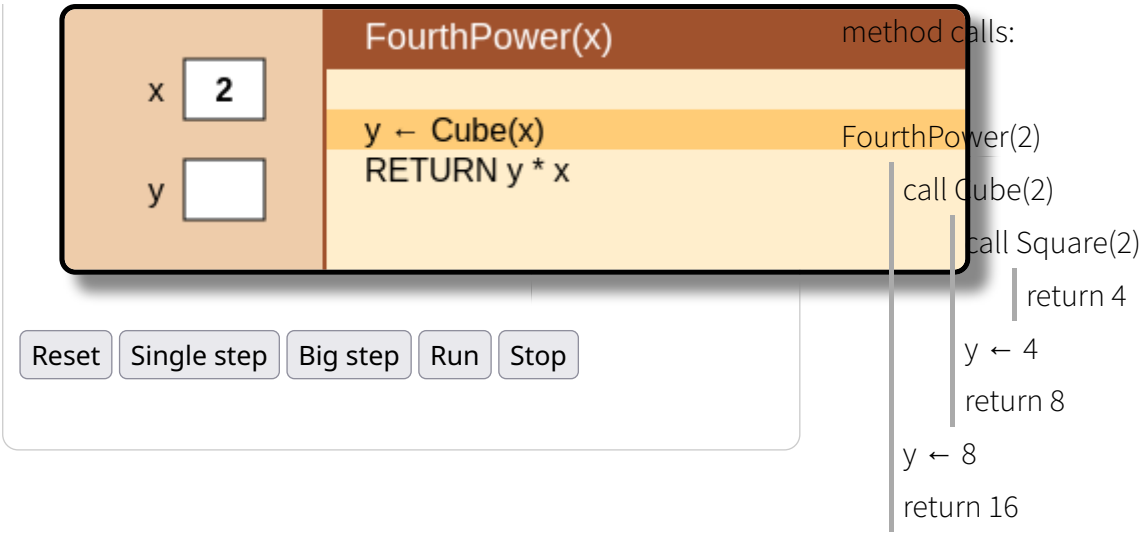
```
y ← Square(x)
RETURN y * x
```

Algorithm FourthPower(x)

```
y ← Cube(x)
RETURN y * x
```

Try executing FourthPower(2):

We can trace the execution of FourthPower, using indentation to show



Similarly, we can trace the execution of `main` in the artificial Java code at right:

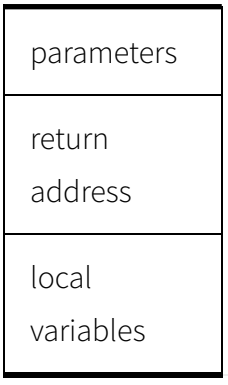
main
- set a to 3
- call `f(4)`
  - print 4
  - call `g(9)`
    - print 9
    - return 55
  - set y to 55
  - print 110
  - return from `f`
- call `g(7)`
  - print 7
  - return 41
- print 41
- return from `main`

```
public static void main(String[] args) {
     int a = 3;
     f(a+1);
     a = g(7);
     System.out.println(a);
}

public static void f(int x) {
     System.out.println(x);
     int y = g(x+5);
     System.out.println(y*2);
}

public static int g(int v) {
     System.out.println(v);
     return v*7 - 8;
}
```

Implementation of methods: the stack

In the above Java program, `main()` calls a method `f()`, which calls a method `g()`. During the execution of a method, the runtime system stores

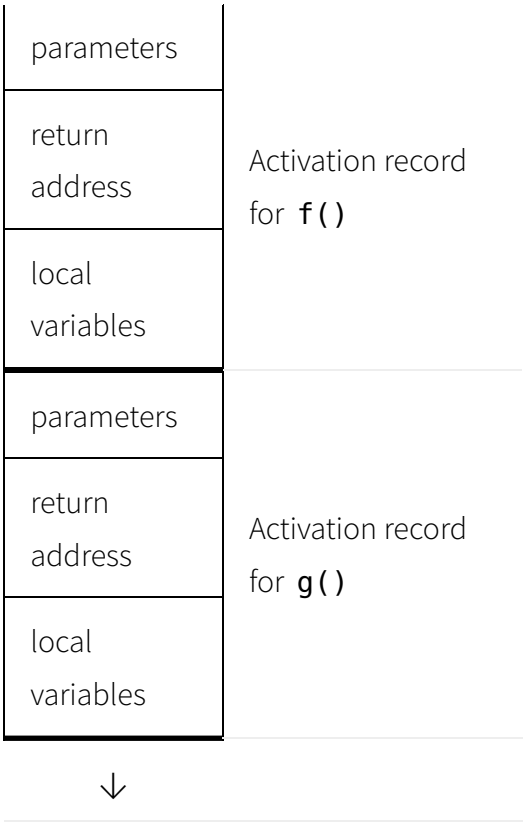
- the values of the parameters passed to the method.
- the *return address*: the location in the program from which the method was called, and to which control will be passed when the method returns.



Activation record for `main()`

- any local variables defined inside the method.
- possible other housekeeping information.

This bundle is called an *activation record* or *stack frame*. There will be one of these for each active method. They are held on the *stack*, which grows and shrinks during execution of the program. The one at right is shown growing downwards.



- When a method is called, a new activation record is added to the stack.
- When the method returns, its activation record is discarded, so that the stack shrinks.

> Objects created with `new` are stored in a different area, called the *heap*.

Recursion

Recursive algorithms use the general strategy of solving large problems by reducing them to smaller ones, and then using the same method on those.

Example: computing powers

We want to compute the function $x^k = x * ... * x$ where k x's are multiplied together. A simple method relies on these two equations:

- $x^0 = 1$
- $x^k = x * x^{k-1}$, if $k > 0$

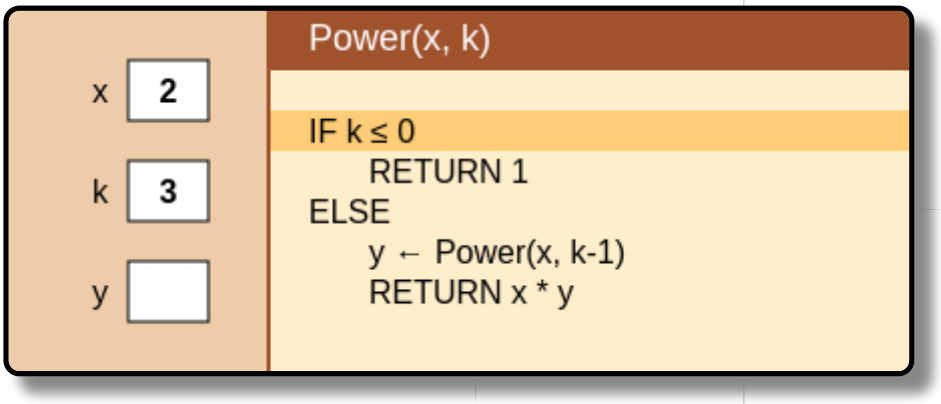
We can translate this directly into a recursive algorithm:

Algorithm Power(x, k)

```
IF k ≤ 0
      RETURN 1
ELSE
```

```
    y ← Power(x, k-1)
    RETURN x * y
```

Try executing Power(2, 3):



Reset | Single step | Big step | Run | Stop

Here is a trace of a call of Power(2, 5):

Power(2, 5)
- call Power(2, 4)
  - call Power(2, 3)
    - call Power(2, 2)
      - call Power(2, 1)
        - call Power(2, 0)
          - return 1
        - return 2*1 = 2
      - return 2*2 = 4
    - return 2*4 = 8
  - return 2*8 = 16
- return 2*16 = 32

Analysis

We shall use the exponent k as the parameter, and count multiplications as our basic step. The recursive power algorithm takes:

- O(k) time

Stack when execution is inside Power(2, 0):

x: 2, k: 5	Activation record for `Power(2, 5)`
return address	
x: 2, k: 4	Activation record for `Power(2, 4)`
return address	
x: 2, k: 3	Activation record for `Power(2, 3)`
return address	
x: 2, k: 2	Activation record for `Power(2, 2)`
return address	
x: 2, k: 1	Activation record for

- O(k) space (the amount of stack required at the deepest point)

return address	`Power(2, 1)`
x: 2, k: 0	Activation record for `Power(2, 0)`
return address	

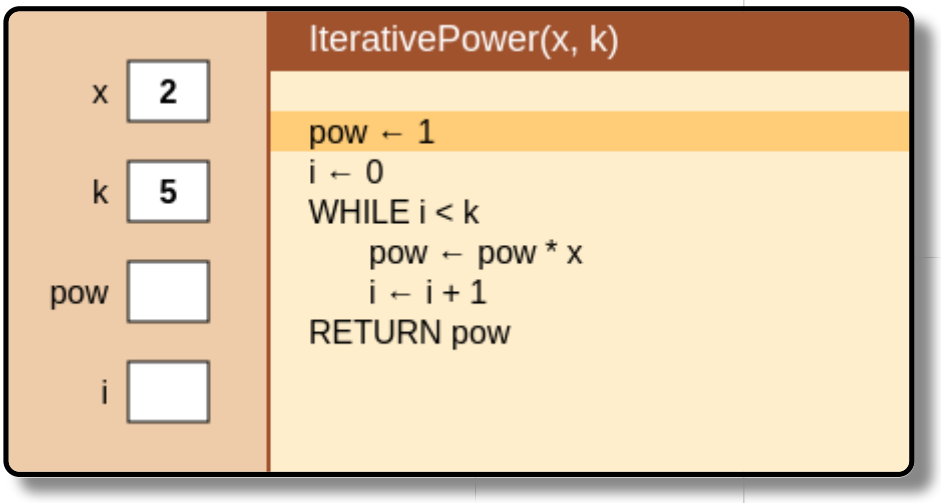
Recursion and iteration

If the last thing a recursive method does is call itself, this call is called *tail recursion*, and that is equivalent to a loop. The above definition of Power isn't tail recursive, because it does a multiplication after calling itself. However we can still transform it into a loop:

Algorithm IterativePower(x, k)

```
pow ← 1
i ← 0
  invariant: 0 ≤ i ≤ k and pow = x^i
WHILE i < k
      pow ← pow * x
      i ← i + 1
RETURN pow
```

Try IterativePower(2, 5):



Reset | Single step | Big step | Run | Stop

Analysis

The iterative power algorithm takes:

- O(k) time
- O(1) space

In this case the iterative algorithm is superior because it avoids using O(n) stack space. However, recursive algorithms may be preferred over iterative versions that use the same order of space but are less clear.

A faster algorithm

We can compute powers more quickly (at the cost of extra space), by using these equations:

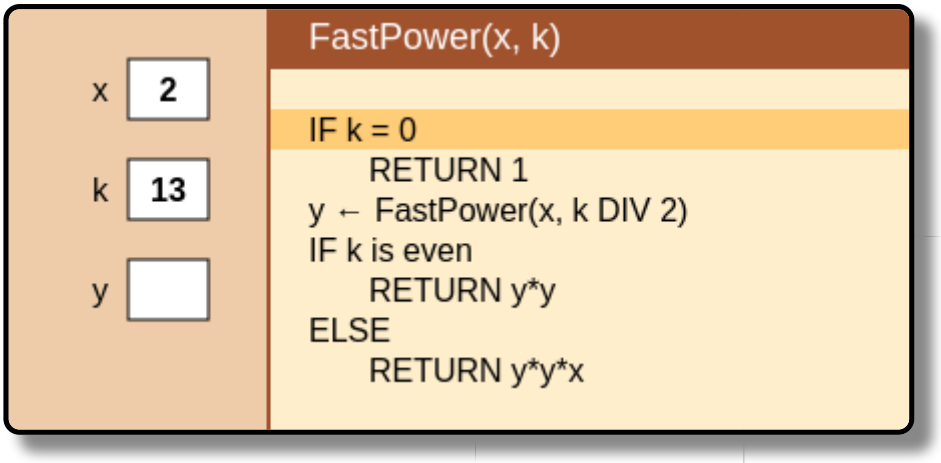
- $x^0 = 1$
- $x^{2k} = x^k * x^k$
- $x^{2k+1} = x^k * x^k * x$

Since the two x^k expressions have the same value, we can compute it once and put it in a variable. We arrive at the following implementation:

Algorithm FastPower(x, k)

```
IF k = 0
        RETURN 1
y ← FastPower(x, k DIV 2)
IF k is even
        RETURN y*y
ELSE
        RETURN y*y*x
```

Try FastPower with x = 2 and k = 13 :



Here is a trace of the execution of FastPower(2, 13):

FastPower(2, 13)
- call FastPower(2, 6)
  - call FastPower(2, 3)
    - call FastPower(2, 1)
      - call FastPower(2, 0)
        - return 1
      - y ← 1
      - return 1*1*2 = 2
    - y ← 2
    - return 2*2*2 = 8
  - y ← 8
  - return 8*8 = 64
- y ← 64
- return 64*64*2 = 8192

At each call, this algorithm halves k. Thus it makes O(log k) nested calls, and takes O(log k) time and O(log k) space.

Divide-and-conquer strategy

A particularly powerful strategy that can be naturally expressed recursively is *divide-and-conquer*:

- If the input is small, compute the answer using a direct

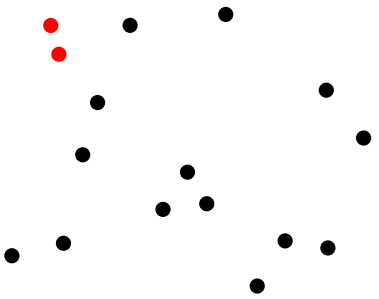
method.

- Otherwise:
  - Split the problem into smaller subproblems
  - Solve each subproblem recursively
  - Combine the solutions

This strategy leads to efficient sorting algorithms, as we'll see in week 7.

Example: closest pair problem

Given n points in 2-dimensional space (described by their coordinates), we want to find the two that are closest to each other. The naive approach to loop over the points, and for each point loop over all the others calculating the distance between the two:



Algorithm SimpleClosestPoints(P)

```
best ← something huge
FOR p in P
        FOR q in P - p
                IF dist(p, q) < best
                        first ← p
                        second ← q
                        best ← dist(p, q)
RETURN (p, q, best)
```

These nested loops will take $O(n^2)$ time.

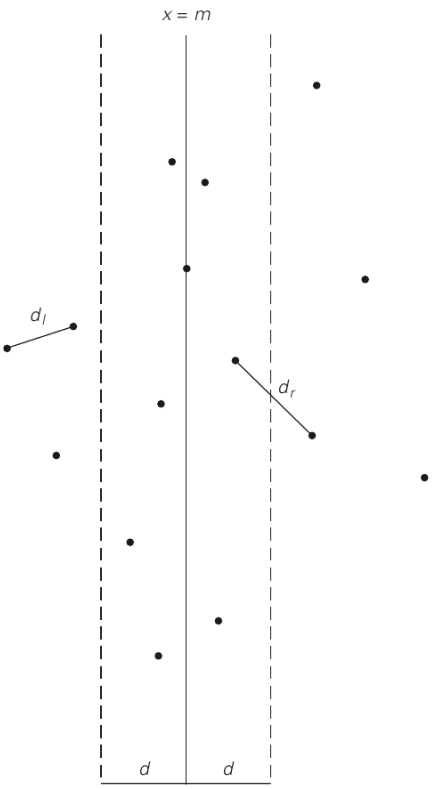
Divide-and-conquer algorithm

The following is a sketch of a faster divide-and-conquer algorithm.

Algorithm FastClosestPoints(P)

```
IF P is small
        RETURN SimpleClosestPoints(P)
m ← x-coordinate dividing P in half
```

$d_l \leftarrow$ FastClosestPoints(left half of P)
$d_r \leftarrow$ FastClosestPoints(right half of P)
$d \leftarrow$ smaller of d_l and d_r
check whether the middle strip has two points that are closer than d
RETURN closest distance



With the appropriate representation, the amount of extra work in each call is proportional to the size of P. From that it can be shown that the total time is O(n log n). Because each subcall takes half as many points, the maximum depth of recursion of O(log n), so the space usage is O(log n).

The calculation of the O(n log n) cost is similar to the analysis of the fast sorting algorithms we'll cover in week 7.

Excessive recursion

Recursion can be an elegant way of expressing a solution, but it is easy to misuse it and get an exponential blowup.

Example: naive Fibonacci function

Consider the problem of finding the nth term in the Fibonacci sequence

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

where each term is the sum of the two before it, i.e.

- $F_0 = 0$
- $F_1 = 1$
- $F_n = F_{n-1} + F_{n-2}$, for $n > 1$

Leonardo Fibonacci (c. 1200 AD)

This translates directly into a recursive function:

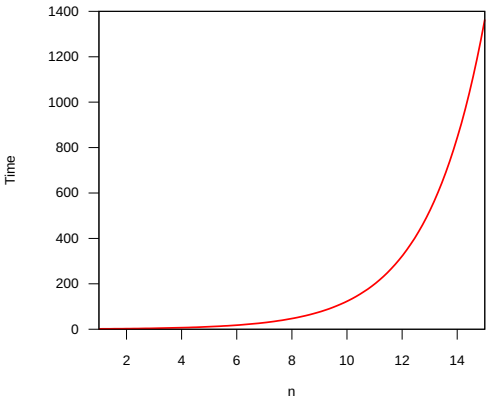
Algorithm Fibonacci(n)

IF n = 0
    RETURN 0
IF n = 1

```
        RETURN 1
    RETURN Fibonacci(n-1) + Fibonacci(n-2)
```

If we trace Fibonacci(4), we see wasteful recalculation:

- Fibonacci(4)
  - call Fibonacci(3)
    - call Fibonacci(2)
      - call Fibonacci(1)
        - return 1
      - call Fibonacci(0)
        - return 0
      - return 1
    - call Fibonacci(1)
      - return 1
    - return 1+1 = 2
  - call Fibonacci(2)
    - call Fibonacci(1)
      - return 1
    - call Fibonacci(0)
      - return 0
    - return 1+0 = 1
  - return 2+1 = 3



Cost of the simple Fibonacci algorithm

In fact this function takes time $O(F_n)$, which grows exponentially.

We can fix this by working up from small values to large ones:

Algorithm FasterFibonacci(n)

```
IF n = 0
    RETURN 0
k ← 1
prev ← 0
x ← 1
```

invariant: $0 \leq k \leq n$ and $x = F_k$ and $prev = F_{k-1}$

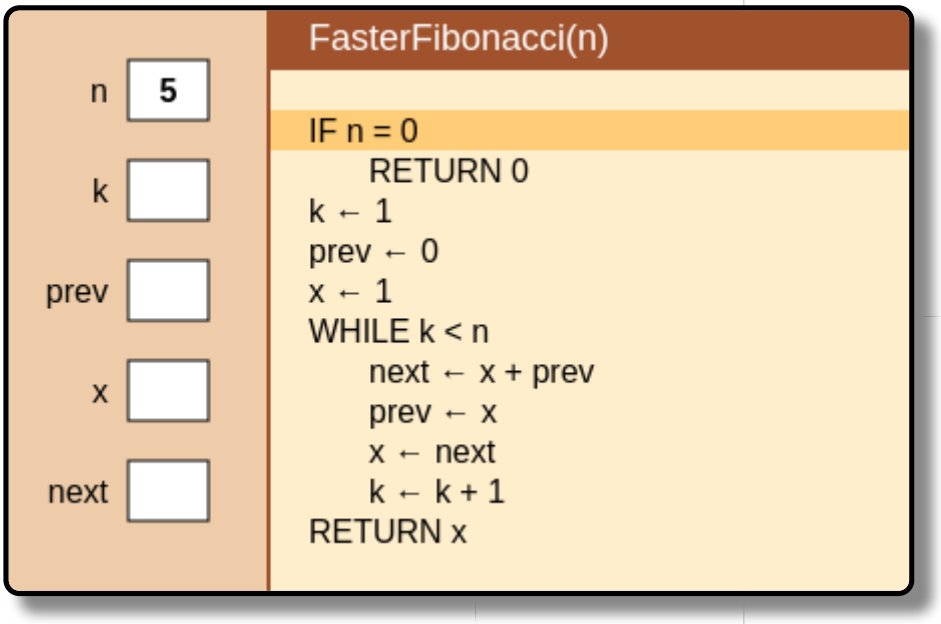
```
WHILE k < n
```

The general strategy solving for small values first and remembering the answers so we don't need to recompute them is known as *dynamic programming*, and leads to efficient algorithms for many problems.

```
        next ← x + prev
        prev ← x
        x ← next
        k ← k + 1
    RETURN x
```

It is obvious from the loop that this version takes time O(n).

Try the faster Fibonacci algorithm:



Reset | Single step | Big step | Run | Stop